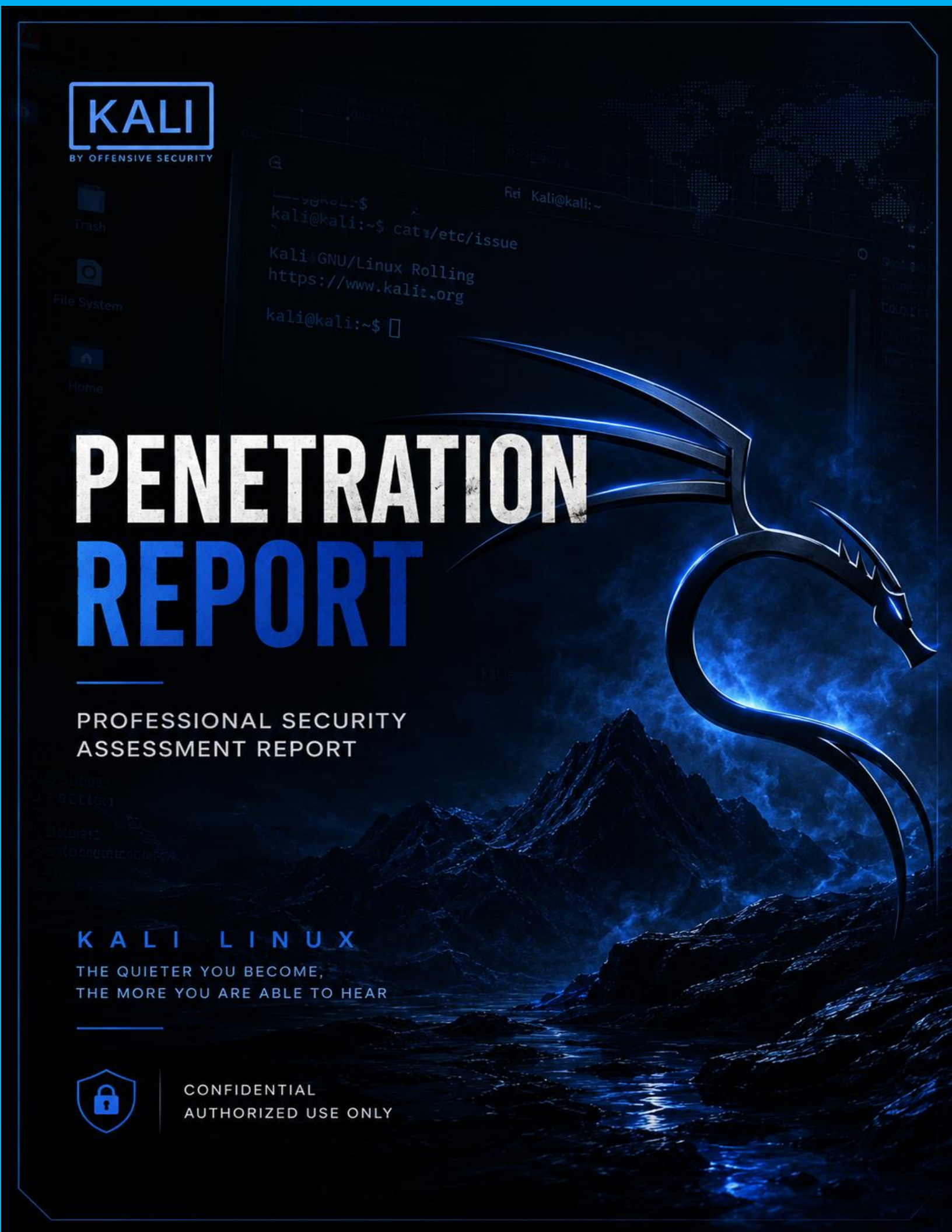

KALI
BY OFFENSIVE SECURITY
Trash
File System
Home
kali@kali:~$ cat /etc/issue
Kali GNU/Linux Rolling
https://www.kali.org
kali@kali:~$
PENETRATION
REPORT
PROFESSIONAL SECURITY
ASSESSMENT REPORT
KALI LINUX
THE QUIETER YOU BECOME,
THE MORE YOU ARE ABLE TO HEAR
CONFIDENTIAL
AUTHORIZED USE ONLY

Assessment scope: 2
Summary:....3
Overview....3
Results....3
Impact....3
Remediation direction 3
Vulnerability writeup 3
Title: 3
Risk rating:....3
Summary: 3
Background: 4
Technical details and evidence: 4
Impact: 6
Remediation advice:....6

Assessment scope:

The assessment was conducted against the Ottergram Daily Challenge lab hosted on the BugForge Labs platform. Testing was limited exclusively to the designated laboratory instance and its associated web application functionality. Target url: https://lab-1787246742315-goy6dh.labs-app.bugforge.io/

Summary:

Overview

The Ottergram application was assessed as part of a controlled penetration-testing exercise to identify security vulnerabilities and evaluate their potential impact. Testing focused exclusively on the authorized BugForge Labs instance and its exposed web application functionality.

Results

The assessment identified a File Inclusion vulnerability within the Ottergram application. Testing demonstrated that user-controlled input was processed by the /api/post/image endpoint, allowing manipulation of the requested file path.

Impact

The File Inclusion vulnerability could allow an attacker to access unintended files on the server, potentially exposing sensitive application data, configuration information, or credentials.

Remediation direction

Strengthen input validation on the file parameter, restrict file access to the intended directory, prevent traversal sequences such as ../, and use an allowlist of permitted files instead of accepting arbitrary paths

Vulnerability writeup

Title:

- File inclusion leading to sensitive files to be revealed

Risk rating:

- Medium

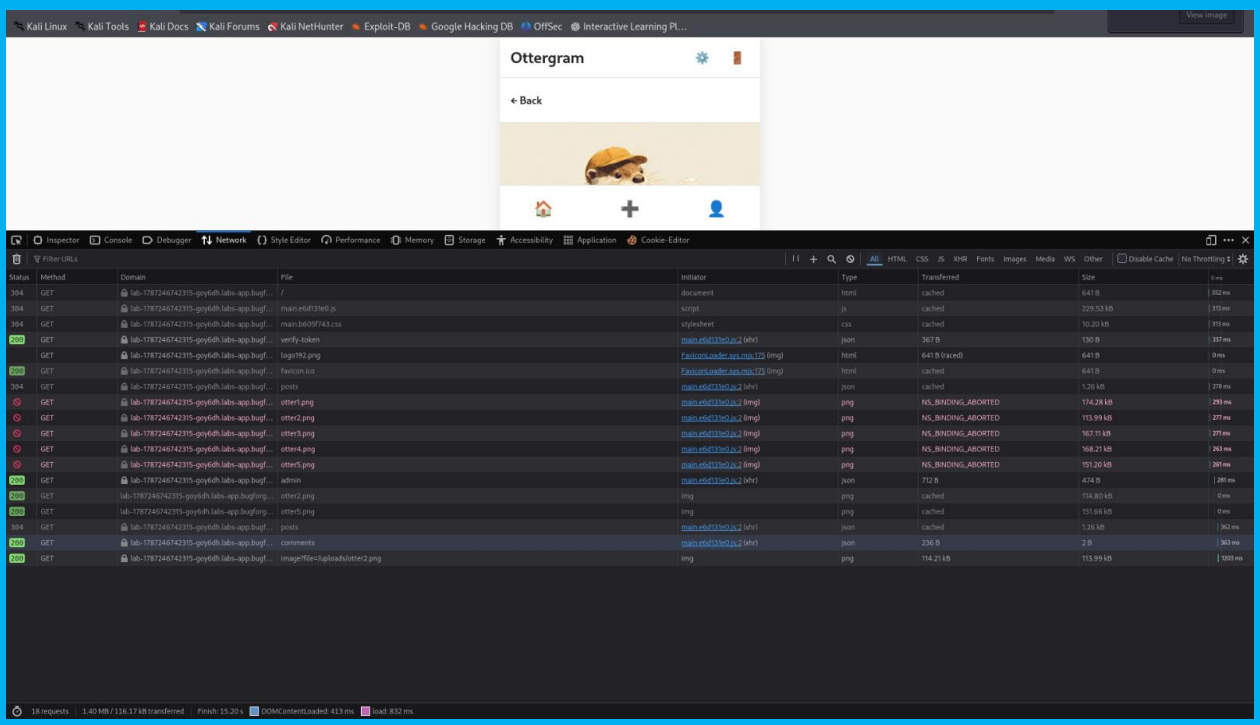
Summary:

During testing, an endpoint was identified through the browser's Network/Inspect tools while the Ottergram application was running. The endpoint accepted a user-controlled file parameter, which appeared to reference files on the server. Testing was performed using a path traversal payload such as ../../../etc/passwd, which confirmed that the parameter could be manipulated and returned a response indicating that the flag was located elsewhere. Further testing with flag.txt successfully returned the challenge flag, demonstrating unauthorized file access.

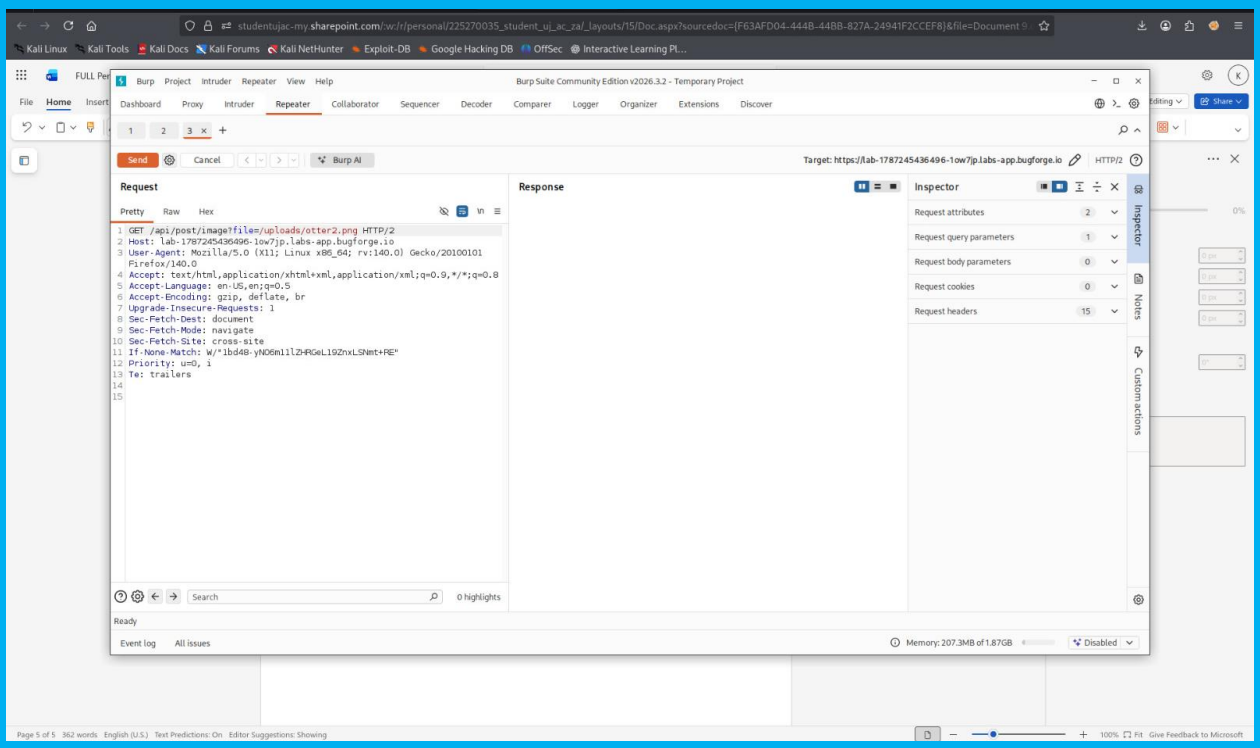
Background:

A path traversal/file inclusion vulnerability occurs when an application does not adequately validate user-supplied file paths. An attacker could potentially use this weakness to access files outside the intended application directory, including configuration files, credentials, source code, or other sensitive information accessible to the application process. This could lead to information disclosure and potentially facilitate further attacks.

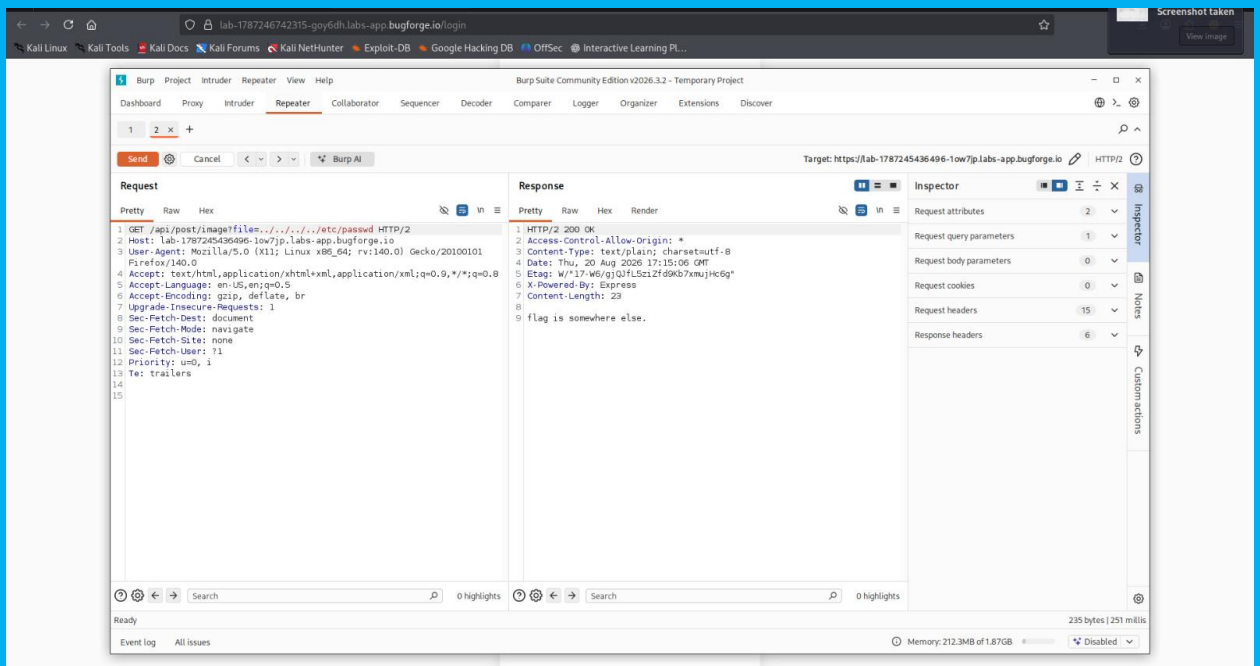
Technical details and evidence:



After authentication, navigation through the application was performed while monitoring the browser's Developer Tools and Network tab. During this process, an interesting endpoint was identified as it was being called by the application.

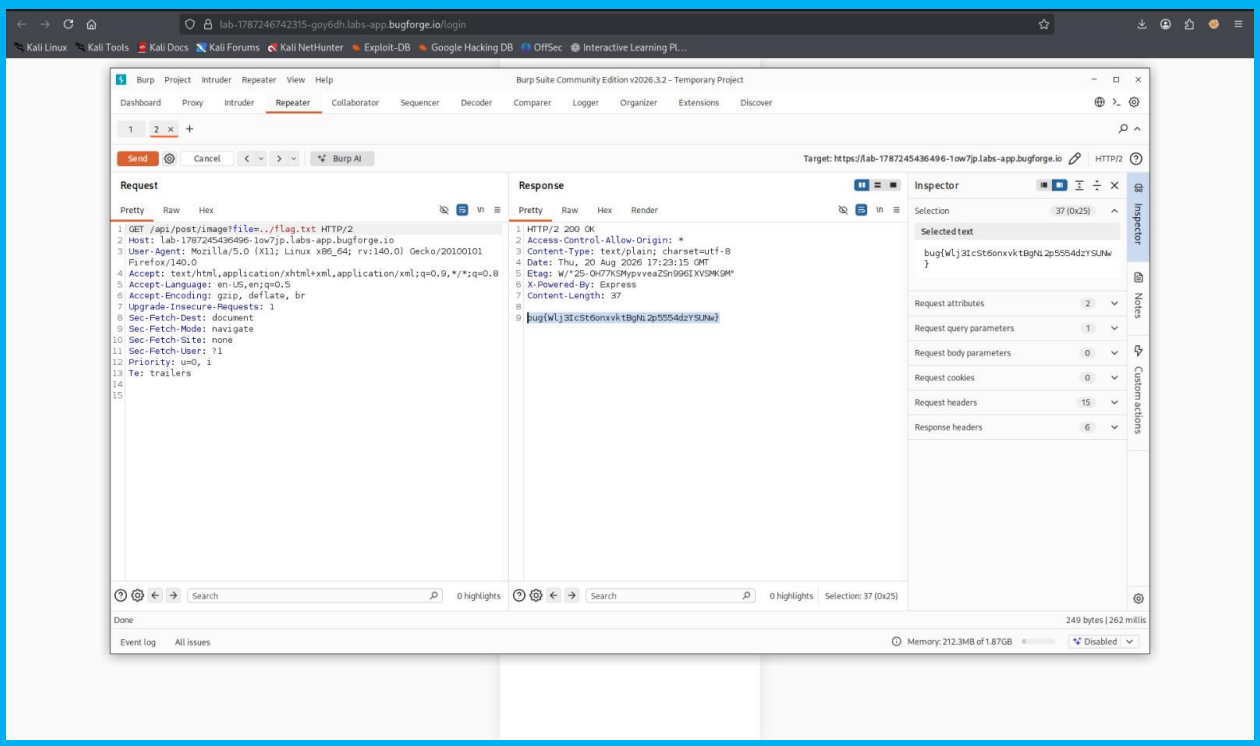


The traffic was intercepted by burpsuite and request was taken to the repeater for further inspection.



The application was tested by manipulating the file parameter with a path traversal payload targeting /etc/passwd. The application responded with a message indicating

that the flag was located elsewhere. This behavior confirmed that the endpoint was processing the supplied file path and was vulnerable to path traversal.



Further testing was performed by modifying the file path to../flag.txt. The application successfully returned the challenge flag.

Impact:

Successful path traversal allowed access to files outside the intended directory. This could potentially expose sensitive files, application configurations, credentials, or other confidential information accessible to the application.

Remediation advice:

Validate and strictly restrict the file parameter to an allowlist of permitted filenames or paths. Prevent directory traversal sequences such as ../, resolve paths securely, and ensure the application can only access files within the intended upload directory.